# Security Protections for KEVIN

Kevin Carriere

2026-05-04

## Contents

**Security for KEVIN** **2**

Website Security Features . . . 2

HTTPS / TLS encryption . . . 2

Reverse proxy (nginx) . . . 2

OAuth2 Proxy authentication layer . . . 2

Firewall (UFW) . . . 2

SSH hardening . . . 2

Brute-force protection (fail2ban) . . . 2

OS / process isolation . . . 2

HTTP Security Headers . . . 2

User Security Features . . . 3

Passwords are hashed via sodium::password_store(). . . . 3

Passwords also required basic, lowest bar security. . . . 4

User must update password on first login. . . . 4

Data Security Features . . . 4

Data building happens outside of server. . . . 4

Writing to db tracked by user. . . . 4

Daily Backups . . . 4

Code-Based Security Features . . . 4

Server Executes Only After Login . . . 4

Dropdown selections must match user-defined permissions . . . 5

User-input must match UI suggestions (multi-value) . . . 5

User input must match suggestions (single value validation) . . . 6

SQL Protections (Injections) . . . 7

Sanitize Text-Based Input . . . 7

Parameterized Queries . . . 7

# Security for KEVIN

## Website Security Features

### HTTPS / TLS encryption

- Let's Encrypt SSL certificate for app.kevinforns.com, auto-managed by Certbot
- Modern TLS protocols enforced (TLSv1.0–1.3, no SSLv3 — protected against POODLE)
- Server-preferred cipher ordering enabled
- Diffie-Hellman parameters (ssl-dhparams.pem) configured for forward secrecy

### Reverse proxy (nginx)

- nginx fronts the Shiny app — public traffic doesn't hit shiny-server directly
- Shiny-server bound to internal port 3838; only reachable via the proxy
- HTTPS termination at nginx layer (port 443), HTTP traffic on port 80

### OAuth2 Proxy authentication layer

- oauth2-proxy running on 127.0.0.1:4180, sitting between nginx and Shiny
- Provides a second authentication factor before users even reach the Shiny login screen
- Adds protection against credential-stuffing and brute-force attacks targeting the app's own login

### Firewall (UFW)

- Default-deny inbound policy
- Only ports 22 (SSH), 80 (HTTP), and 443 (HTTPS) open to the internet
- Port 3838 (raw shiny-server) is not exposed externally — only locally on the droplet
- IPv6 traffic firewalled with the same rules

### SSH hardening

- Root login disabled (PermitRootLogin no)
- Password authentication disabled (PasswordAuthentication no)
- Key-based authentication required (PubkeyAuthentication yes)

### Brute-force protection (fail2ban)

- fail2ban service running, monitoring authentication failures and auto-banning suspicious IPs

### OS / process isolation

- shiny-server runs apps as the unprivileged shiny user (run_as shiny in config)
- App processes can't touch other users' files or system-level resources

### HTTP Security Headers

### Scrict-Transport-Security

- max-age=63072000; includeSubDomains

#### Content Security-Policy

- default-src 'self'
- script-src 'self' 'unsafe-inline' 'unsafe-eval'

*Note: inline and eval is required for Shiny to function.*

*Long term suggestion by Claude: Need to fork or wrap shiny-server to inject nonces, which is a real engineering project.*

- style-src 'self' 'unsafe-inline'
- img-src 'self' data:
- frame-ancestors 'none'

#### X-Frame-Options

- deny

#### X-Content-Type-Options

- nosniff

#### Referrer-Policy

- strict-origin-when-cross-origin

#### Inactivity Timeout

- 20 minutes of inactivity, time out via inactivity.js

## User Security Features

#### Passwords are hashed via sodium::password_store().

From the help file:

> The password_store function returns an ASCII encoded string which contains the result of a memory-hard, CPU-intensive hash function along with the automatically generated salt and other parameters required to verify the password. Use password_verify to verify a password from this string.

```
new_hash <- sodium::password_store(new_pw)


password_verify(current_hash, old_pw))
```

> Wrapper that implements best practices for storing passwords based on scrypt with a random salt.

**Passwords also required basic, lowest bar security.**

- at least 8 characters, at least 1 uppercase, at least 1 lowercase, 1 symbol, at least 1 number.

**User must update password on first login.**

## Data Security Features

**Data building happens outside of server.**

- The sensitive data is built in pipeline, out of server. Only selected columns are passed to the datasets that build the routes and voters that sit serverside.

**Writing to db tracked by user.**

- All input is also tracked on who wrote the input.

```
interaction_fields <- list(
      campaigner       = credentials()$info$user,
```

**Daily Backups**

- Daily back-ups onto my computer (not server) at 9:00am. Keeps the last 30 days worth of backups.

## Code-Based Security Features

**Server Executes Only After Login**

```
req(credentials()$user_auth)
```

**req()** See the help file for shinyauthr::loginServer.

- "use req to only render results when credentials()$user_auth is TRUE"

```
  searchable_voters <- reactive({
    req(credentials()$user_auth)
    allowed_precincts <- routes_raw |>
      dplyr::filter(Scenario %in% user_scenarios()) |>
      dplyr::distinct(Precinct) |>
      dplyr::pull(Precinct)
    voters_raw |> dplyr::filter(Precinct %in% allowed_precincts)
  })
```

**Example** Data building serverside is not executed until someone logs in. Basic auth check is used on:

- user_role()
- searchable_voters()
- voter_data()
- suggested_routes()
- hiding the login UI and showing the app.

### Dropdown selections must match user-defined permissions

**userinput %in% serverlist** %in% checks that the input provided by the user matches what they have already been provided access. Shiny's UI is simply a pretty presentation of 'suggested' inputs, but a user could try and push non-suggested inputs through _____. This checks based on their access, ensuring that the only inputs allowed are ones predefined for them.

```
scenario_access <- list(
  ns    = c("Specific", "Routes", "Listed", "Here")
)

user_role <- reactive({
  req(credentials()$user_auth)
  user_base$role[user_base$user == credentials()$info$user]
})

user_scenarios <- reactive({
    scenario_access[[user_role()]]
})

req(input$scenario %in% user_scenarios())
```

**Example** %in% based on role-access is also used to determine the filtering of data within the inputs of the user.

- searchable_voters()
- my_scenarios
- voter_data()

### User-input must match UI suggestions (multi-value)

In addition, we use %in% to ensure that if the options are Yes or No, the user did not input Banana.

```
checkboxGroupInput(
                    "specialvote_levels",
                    "Include:",
                    choices = c(
                      "Yes (Voted)" = "Yes",
                      "No (Did not vote)" = "No"
                    ))
```

```
    if (isTRUE(input$filter_special) && length(input$specialvote_levels) > 0) {
      out <- out |>
        dplyr::filter(
          nsse_2026_voter %in% input$specialvote_levels |
            (is.na(nsse_2026_voter) & "NA" %in% input$specialvote_levels)
        )
    }
```

**Example** If the user passes TRUE in input$specialvote_levels, the filter silently hits false and returns nothing.

### User input must match suggestions (single value validation)

Similar to above, but instead of filtering, we also use it to validate most of the inputs provided by users.

```
validate_choice <- function(input_value, allowed) {
  if (is.null(input_value)) return(NA_character_)
  if (input_value %in% allowed) return(input_value)
  NA_character_
}

validators <- list(
  had_conversation_choice = c("Yes", "No")
)

were_there_val <- validate_choice(
      input$were_there,
      validators$were_they_there_choice
    )
fields <- list(
      were_they_there = were_there_val
      )

insert_contact(con, vid,
               input_by = credentials()$info$user,
               fields   = fields
               )
```

**Example** If the input is not what is permitted to be chosen, the input is set to null and nothing is written for that input value into the database.

Validated input from user includes:

- campaigner_val
- left_material_val
- were_there_val
- had_convo_val
- support_rating_val
- concern1_val

- concern2_val
- concern3_val
- addl_support_val
- new_left_material_val
- new_support_rating_val
- new_concern1_val
- new_concern2_val
- new_concern3_val
- notes_val
- know_them_val

## SQL Protections (Injections)

### Sanitize Text-Based Input

**sanitize_user_input()** We implement a quick clean on the input provided by the user. We originally were using shinyvalidinp::validinp_character(), but found that the regex of permissive cases was overly burdensome, and opted for a more straightforward solution.

```
sanitize_user_input <- function(x) {
  if (is.null(x)) return(x)

  x_char <- as.character(x)

  x_char <- trimws(x)
  x_char <- gsub("[\x01-\x08\x0B\x0C\x0E-\x1F\x7F]", "", x_char, perl = TRUE)
  # I had this at one point, but have gotten rid of it in favor of the gsub.
  #x_char <- gsub("[[:cntrl:]]", "", x_char)

  x_char <- gsub("[\t\r]", " ", x_char)
  x_char <- htmltools::htmlEscape(x_char)

  # Preserve NAs from original
  x_char[is.na(x)] <- NA_character_
  x_char
}
```

- Set the input as a character string.
- Trim the white space.
- Remove non-printing control characters
- Set enters & tab to space.
- If after this, nothing is left, set NA.

### Parameterized Queries

User inputs are transmitted separately as parameters into the SQL input. As a result, the database treats parameters strictly as values, regardless of their content, and does not treat them as SQL code.

```
dbExecute(con,
      "INSERT INTO new_contacts
       (voter_id, first_name, last_name, address, precinct, route, stop_order,
        scenario, recorded_by, timestamp)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
      params = list(new_vid, first_name, last_name, address,
                    as.integer(precinct), as.integer(route), as.integer(stop_order),
                    scenario, recorded_by, now_ts))
```

**Example** Used in all writes:

- insert_new_contact(){}
- insert_additional_info(){}
- append_who_knows(){}
- append_additional(){}
- get_additional_notes(){}
- update_user_password(){}
- password_change_server(){}